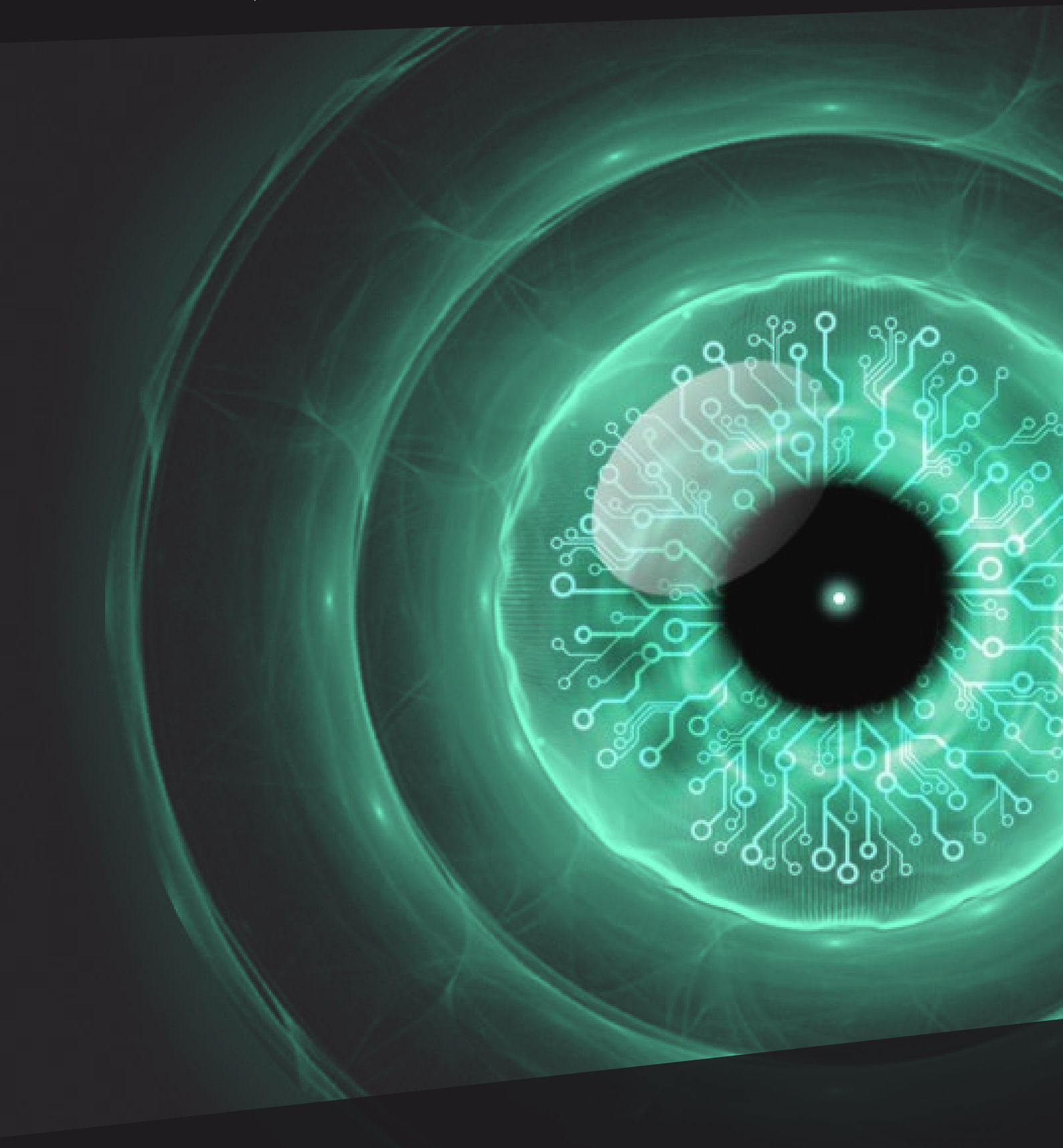


ANYVERSE CAMERA SENSOR SIMULATION V2.0

Physically-based sensor simulation to shorten development lifecycles.

CONTENTS

1. INTRODUCTION

> *Developing perception systems is not an easy task. We are talking about systems that need to understand what they see in the real world and react accordingly. But, How do they see the world? How do you teach a machine what the real world is and interpret it?*

Modern perception systems use all kinds of sensors to see the world like radar, lidar and of course optical cameras that are the ones that most faithfully mimic the human eye, which coupled with the brain, form the most advanced perception system that exists to date. On the other hand, to implement the perception system "understanding" they use deep neural networks trained for different purposes, like object detection, object segmentation or depth estimation.

No matter what the problem is, neural networks need data, lots and lots of data. But not any data, it has to be as varied as possible to avoid bias in the system and other domain shift problems.

Getting real-world data for your hungry system is not easy. You have to take thousands of pictures and curate them, which requires infrastructure and organization, it can be a separate project in itself. If that is not enough, just images are not enough either for neural networks to learn.

No matter what the problem is, neural networks need data, lots and lots of data. But not any data, it has to be as varied as possible to avoid bias in the system and other domain shift problems.

While training, you need to tell the neural network what is that it's seeing, and for that, you need to tag and annotate every single image with ground truth. One more, time-consuming, and often not a very accurate task. When you thought you were done, it turns out your system is not performing well, so you need more training and yes, more data.

An alternative to this endless loop is to use synthetic data. You create a synthetic 3D scenario and render thousands of images based on it adding automatic variability and generating all the ground truth info you need at the same time. Fair enough, problem solved. Not quite.

When you train deep neural networks with synthetic data you have to make sure that they will be able to perform when facing the real world and understand it as well as the synthetic data. How well your network generalizes real-world images from synthetic images, is key for your system's success.

For that, you need to faithfully simulate the behavior of real cameras when generating synthetic images.

Anyverse™ is a high-fidelity synthetic data solution featuring a physics-based synthetic image render engine to produce very realistic images and sequences of unprecedented quality.

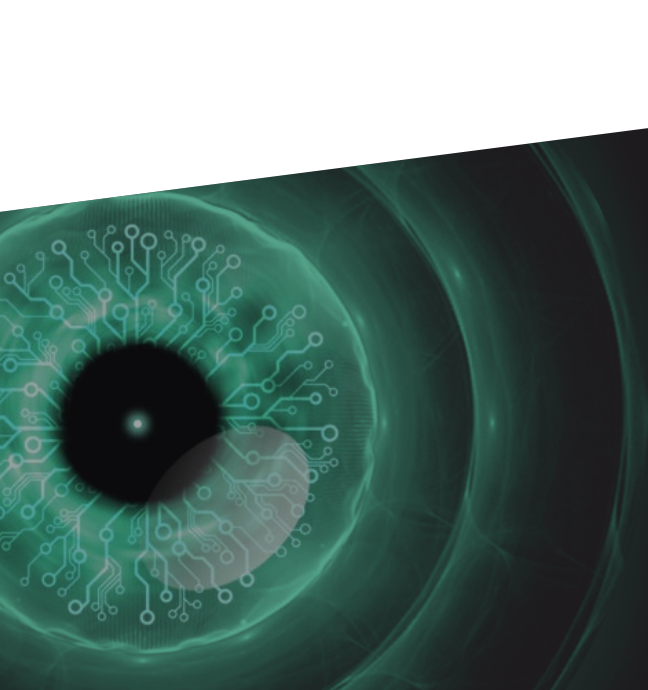
How well your network generalizes real-world images from synthetic images is key for your system's success.

The engine uses an accurate light transport model and provides a physics description of lights, cameras and materials. This allows for a very detailed simulation of the amount of light that is reaching the camera sensor.

Equally important is the simulation of the camera sensor itself, i.e. how the light coming from the scene is converted into the final color image. Different academic papers demonstrate that a machine learning model, based on deep neural networks, trained on a synthetic dataset generated considering camera sensor effects, performs in general better than if the effects are not present. You can check these papers on the subject [1][2].

In this eBook, we focus on the camera sensor simulation. What you need to implement it, what parameters and knobs you can have under your control, what are their effects on the final images and of course what are the benefits of having sensor simulation as part of your perception system development process.

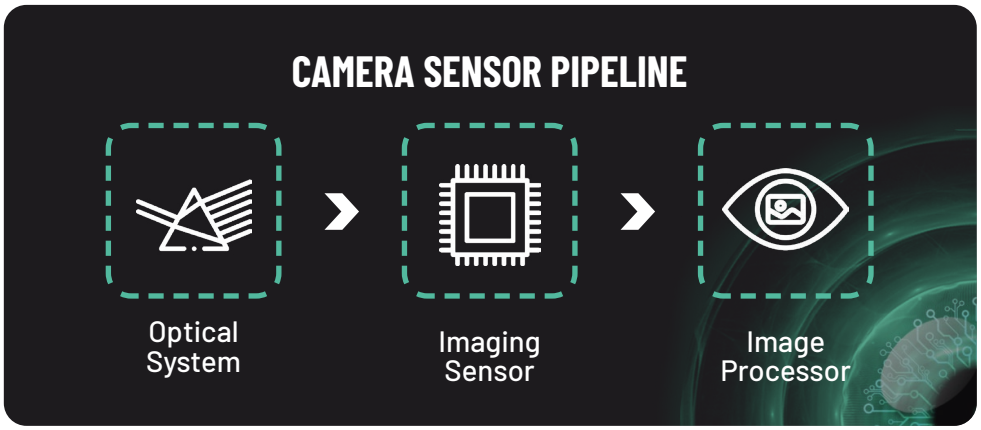
We assume some knowledge about digital cameras and specifically about CMOS camera sensors. But if you are new to the subject or want to review that knowledge, you can find a very basic introductory tutorial by the photographer's community in this reference [3].



Angel Tena
Chief Technology Officer

2. CAMERA SENSOR PIPELINE OVERVIEW

Let's start with the basics: what does a generic camera sensor look like? It basically consists of three main blocks.



The light coming from the scene goes through the optical system. Then light reaches the sensor surface where it is converted into voltage, and finally into digital value. All this happens in the imaging sensor. The digital value coming from the imaging sensor is converted into a final RGB image in the image processor block.

The key point to emphasize is that we account for the electromagnetic spectrum when simulating light energy. For instance, light sources in Anyverse™ emit using a characteristic spectrum profile that depends on the type of light source (LED, incandescent bulb, sky, etc). The materials' behavior is also wavelength-dependent, this allows for typical effects like dispersion. This feature is key and absolutely necessary for an accurate camera sensor simulation.

> Companies developing new generation perception systems need to know how sensitive their deep learning perception models are to different sensors and configurations, to make the right decisions fast. They need an agile and cost-effective way to run dozens or hundreds of experiments with reliable data.
>
> Víctor González | Founder & CEO at Anyverse

3. CAMERA SENSOR PIPELINE DETAILED

Now is when the fun starts, let's play with the light! As we said, if the light is physically characterized we can see (and simulate) how it behaves interacting with the different components of the pipeline to create an image.

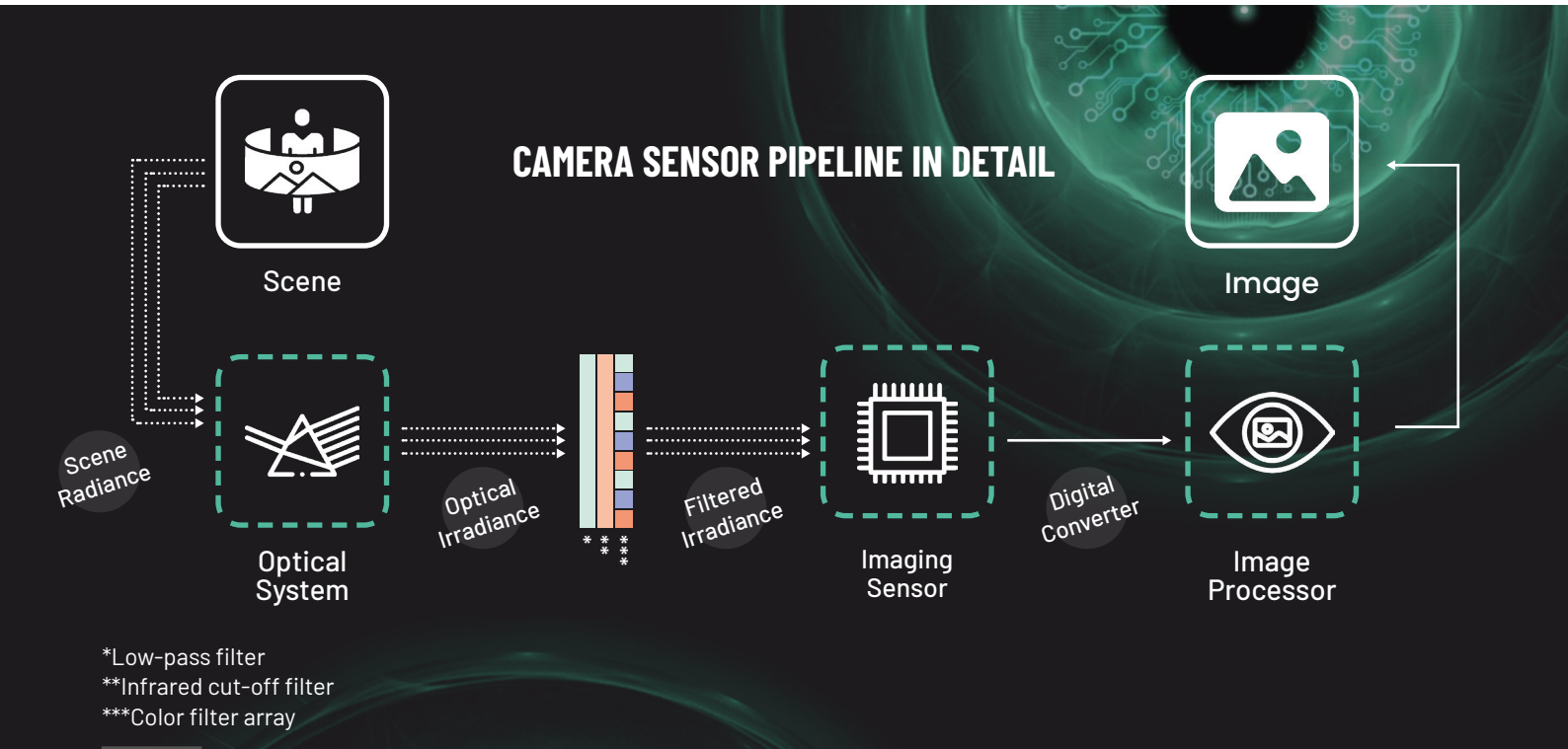
Typical camera mountings include some filters between the optics and the sensor surface. These filters work in the spectrum domain performing different transformations on the incoming light.

In the picture below you can see a common set of filters used. The low-pass filter and the near-infrared cut-off filter remove part of the spectrum in the near-infrared range because our eyes are not sensitive to them. It is worth mentioning that the range of the electromagnetic spectrum simulated is not only the visible but the near-infrared as well.Some materials behave differently in the near-infrared part of the spectrum and Anyverse™ does account for this. Specifically the range of the electromagnetic spectrum simulated is [380-1000] nanometers. The ability of simulating the behavior of light and materials in the near-infrared part of the spectrum is becoming increasingly critical for some applications like for instance in-cabin monitoring systems.

The ability of simulating the behavior of light and materials in the near-infrared part of the spectrum is becoming increasingly critical for some applications like for instance in-cabin monitoring systems.

The color filter array (CFA) is another key component of the camera mounting. A light photodetector detects light intensity with no wavelength specificity, therefore color information can't be separated. The CFA separates color information before reaching the sensor's surface. We will see more details about the filters in a later section.

In Anyverse™, the scene energy is collected in the aforementioned part of the electromagnetic spectrum, that's why the filtering process described above is possible.



3.1 OPTICAL SYSTEM

Light gets into the camera through the lens. In Anyverse™ we do a geometric simulation of the optical system using our ray tracing technology.

We can simulate extreme optics distortion as shown in the picture below.

Fisheye lens

Even though in terms of simulation of the optical system we can use approximations like the thin-lens or thick-lens models, in Anyverse™ we can also model complex system of lenses that are described using a prescription.

The description of the system of lenses is a spreadsheet-like document where one row represents one surface of the system. This is a very standard way of representing a system of lenses.

For example, in Zemax© the LDE (Lens Data Editor) provides, using this format, the information that describes the system of lenses. See image below.

Lens Data Editor

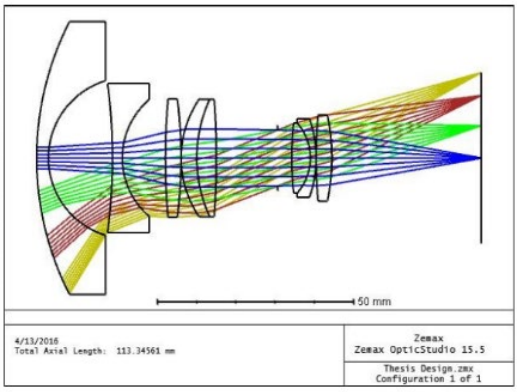
Edit Solves Options Help

	Surf:Type	Comment	Radius		Thickness		Glass
OBJ	Standard		Infinity		Infinity		
STO	Standard		Infinity		0.000000		
2	Standard		Infinity		10.000000		
3	Standard		100.000000		5.000000		BK7
4	Standard		-111.467299	V	20.000000		
5	Coordinate B..				0.000000		-
6	Standard		Infinity		0.000000		MIRROR
7	Coordinate B..				0.000000		-
8	Standard		Infinity		-80.734561	M	
IMA	Standard		Infinity		-		

Zemax© Lens Data Editor

In the image below a prescription for a fisheye system is shown. This design was created with Zemax©.

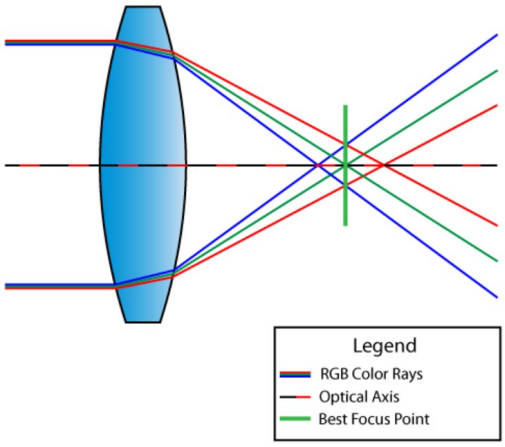
F/# = 2.84					
f = 15.99 mm					
HFOV = 90 Degrees					
Back Focal Distance = 38.09 mm					
Surface No.	r	d	N_d	V_d	Clear Aperture
1	76.461	3.000	1.69680	55.4	69.5
2	20.615	16.324	-	-	39.4
3	-144.269	2.500	1.69680	55.4	38.0
4	19.579	10.993	-	-	29.7
5	168.743	4.000	1.51112	60.4	30.0
6	-118.106	0.000	-	-	30.0
7	26.797	4.000	1.51680	64.2	30.0
8	35.702	5.000	1.64769	33.8	30.0
9	-198.347	15.645	-	-	30.0
10 (Stop)	-	3.575	-	-	14.2
11	83.789	4.715	1.49700	81.6	17.6
12	-13.597	1.500	1.80518	25.4	17.8
13	-30.051	0.000	-	-	19.8
14	123.727	4.000	1.51680	64.2	22.0
15	-52.823	-	-	-	22.0



Fisheye prescription

Simulating a complex system of lenses allows us to account for chromatic aberration artifacts. These artifacts come up naturally when you have a hyperspectral render combined with the ability to simulate a system of lenses. Video games and other non-spectral renders mimic the effects of chromatic aberration as a postprocess on the final image, which is, of course, just a low-fidelity approach.

There are two types of chromatic aberration, axial (longitudinal) and lateral aberration.



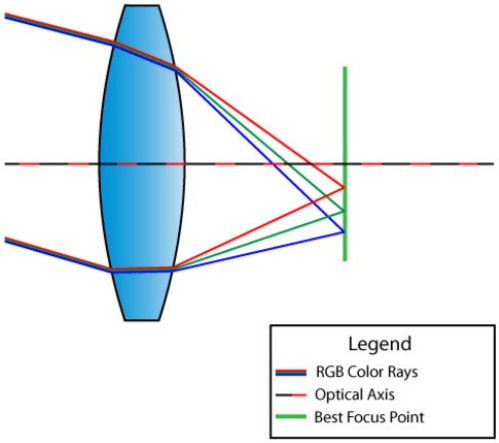
Axial chromatic aberration
(image by NASIM MANSUROV from photographylive.com)

Axial aberration occurs when different wavelengths of color do not converge at the same point after passing through a lens. This is illustrated in the image to the left.

Axial aberration produces blurring in the images. Below, to the left, we show a rendering of a scene where the system of lenses from the prescription shown earlier was used. Inside the red circle a crop of the same image rendered without chromatic aberration is shown. The image inside the circle below is sharper than the original as no chromatic aberration was simulated.

Axial chromatic aberration

Lateral chromatic aberration



Lateral chromatic aberration
(image by NASIM MANSUROV from photographylive.com)

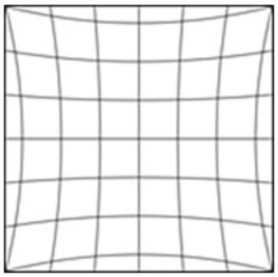
Lateral aberration occurs when different wavelengths of color coming at an angle focus at different positions along the same focal plane, this is illustrated in the image to the left.

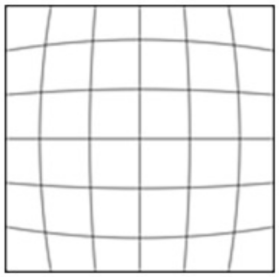
Lateral aberration produces fringing in the images.

Above to the right is a rendering of a scene where the system of lenses from the prescription shown earlier was used. In the red circles we can see a close up of the parts of the image where the fringing is very evident.

In Anyverse™ we have also the ability to simulate different forms of optical aberration, like for example barrel and pincushion distortions. Those types of distortions are normally characterized by a polynomial with different coefficients that depend on the distance of the pixel to the center of the image. Those coefficients are found by means of calibration.

Pincushion Distortion





Barrel Distortion

There are several distortion models supported, with OpenCV the most requested and used. In Anyverse™ we support both OpenCV and OpenCV Fisheye distortion models.

Another important effect caused by the optics is lens shading. The view angle allows rays that are reaching the corners of the sensor to travel more distance, hence there is a decrease in the energy reaching those areas. Since focal length affects the view angle, this parameter has an impact on lens shading too.

Below, an image from Anyverse™ showing the lens shading effect.

Lens shading

Lens blurring

We can simulate lens blurring too, also known as depth of field. Again the ray-tracing technique allows us to compute an accurate depth of field effect. Focal length and aperture are the parameters that have a high impact on this effect. The picture above shows this effect amplified for the sake of visualization.

Lens imperfections might lead to different unwanted artifacts. In Anyverse™ glare and diffraction artifacts are simulated. Both of them are simulated using a phenomenological approach, especially the diffraction artifacts that are due to the interference effect of energy when it is regarded as an electromagnetic wave.

Glare can appear as hazy or overexposed areas in an image, often surrounding bright light sources. Lens imperfections can scatter light in a way that creates unwanted brightness and reduces image contrast, leading to glare. Below is a series of three images that represent the glare from not having it to two levels of intensity.

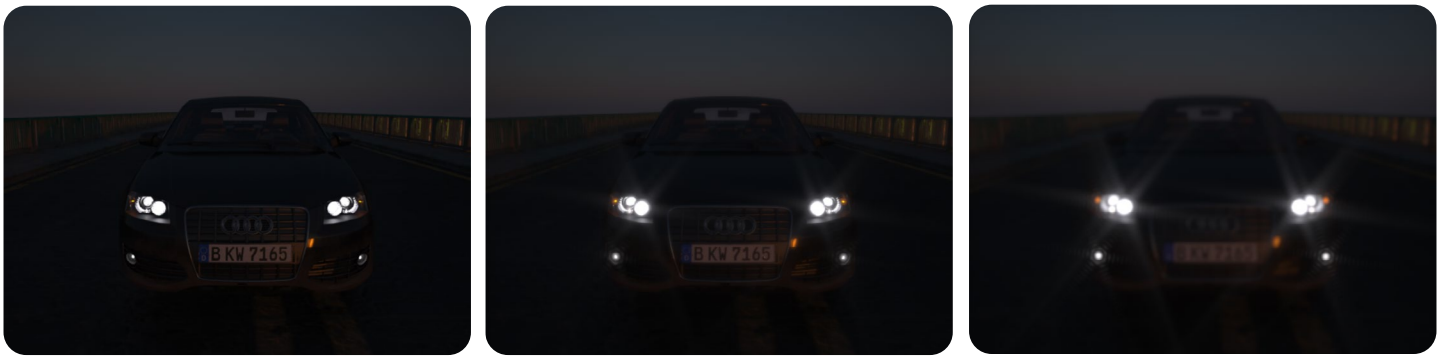
No glare

Glare intensity 1

Glare intensity 2

Diffraction artifacts manifest in images as patterns characterized by a starburst appearance. Diffraction refers to the phenomenon where waves exhibit a slight bending as they encounter small obstacles and disperse as they pass through narrow apertures. When light traverses into the camera through a small aperture, particularly at a reduced focal length, it undergoes bending around the aperture's blade edges, giving rise to the star-like visual effect.

The quantity of rays within each starburst is directly associated with the number of aperture blades present in the lens. Again below there is another series of three images that represent the diffraction around the headlights of the car from not having it to two levels of intensity.



No diffraction **Diffraction intensity 1** **Diffraction intensity 2**

In the image below we show a combination of both lens artifacts, glare and diffraction.

Representation of the combination of both lens artifacts, glare and diffraction


DENTAL CARE
POLICE

3.2 THE IMAGE SENSOR

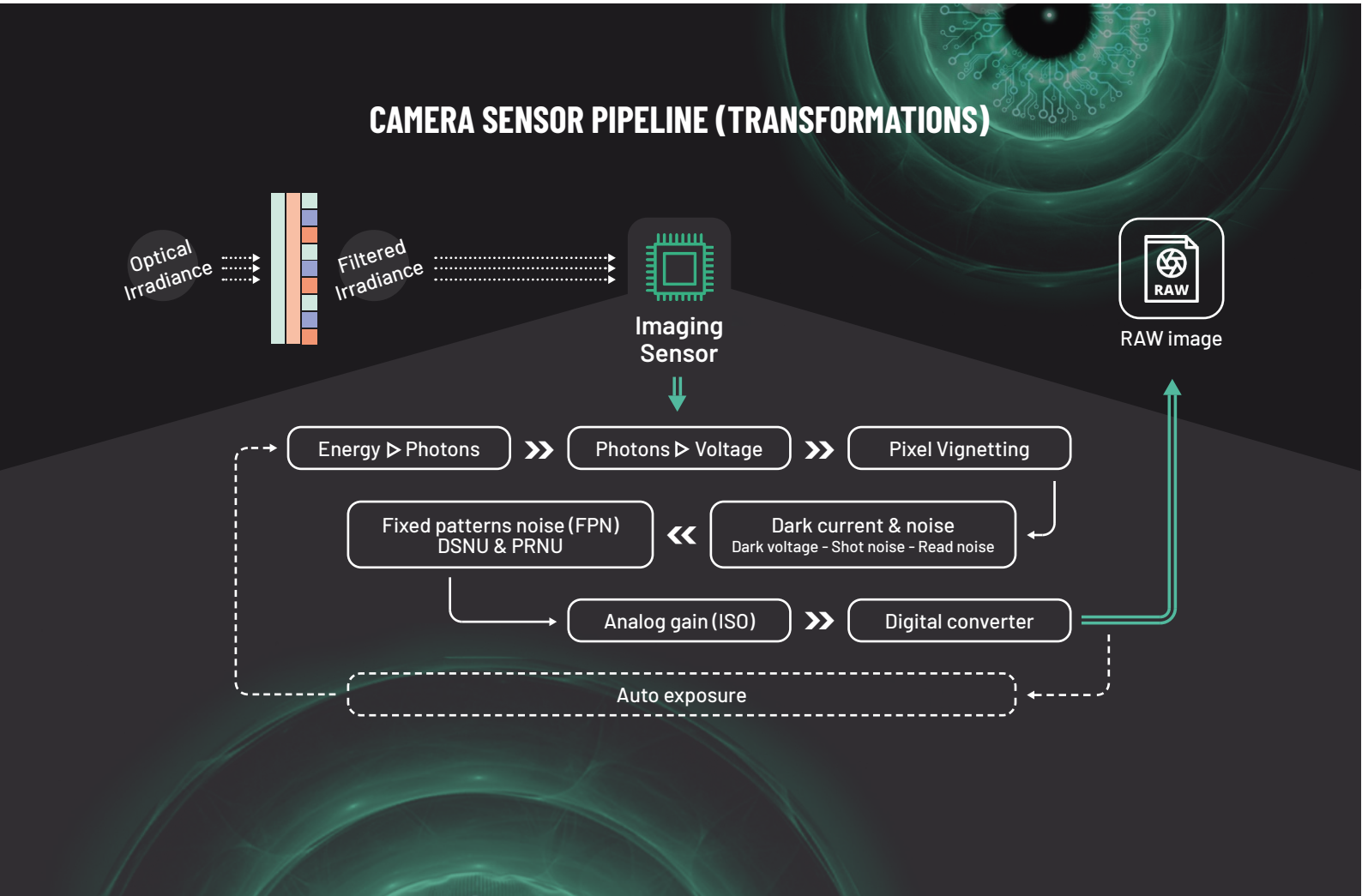
> *The next step of the light in its travel to become a picture, is the imaging sensor.*

The core of the system, the electronic device that substitutes the film in classic analog cameras. It basically transforms the energy from the photons reaching it into voltage that, in turn, converts it into digital values that represent every pixel in the final image.

To simulate all the processes that happen inside an imaging sensor, you don't need to simulate all the electrical circuitry and its components (transistors, capacitors, etc.). It is similar to when you want to simulate the dynamics of fluids, you don't need to simulate every single fluid molecule.

The most important phenomena occurring in an imaging sensor can be described with mathematical functions that transform the information at different stages to get the final color digital values.

In the infographic below we describe the most important transformations that are modeled by the Anyverse™ camera sensor simulator.



The spectral radiance coming from the scene that we have after going through the optics system, is converted into spectral irradiance at the sensor surface. This spectral irradiance comes in terms of watts per square meter per nanometer. This is the amount of energy per wavelength that is reaching the sensor's surface, and this is exactly the information that we have at the input of the imaging sensor block.

Now let's see what's happening at every imaging sensor stage.

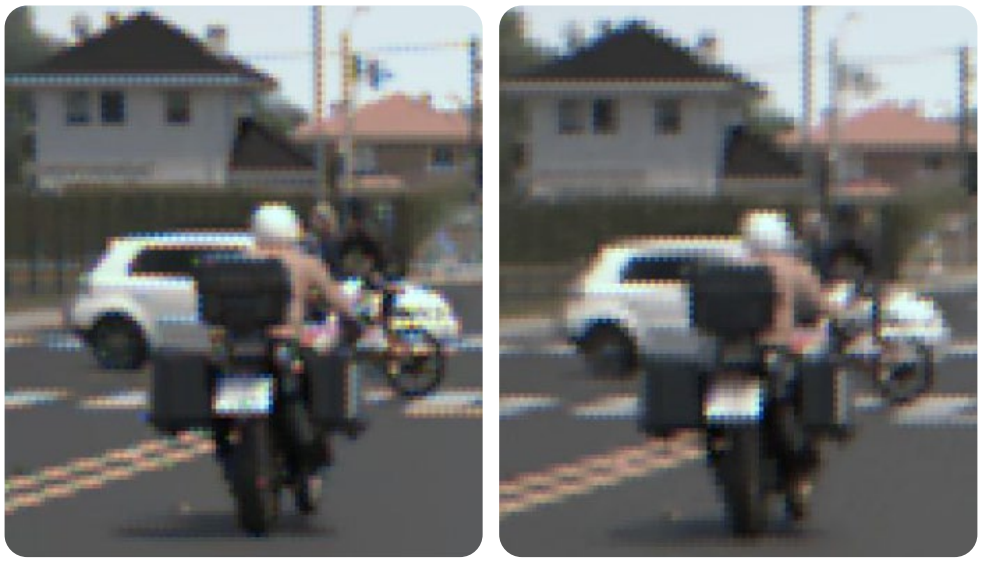
THE FILTERS

Not all the information coming in the light beam is necessary to create the final image, some information is discarded and other is transformed. That's why the spectral irradiance needs to go through different filters before reaching the photodetectors on the sensor surface. The low-pass filter, the infrared cut-off filter and the color filter array.

LOW-PASS FILTER

This is an antialiasing filter that removes Moiré artifacts [4]. Some cameras don't have this filter, that's why in Anyverse™ we provide the ability to turn it on or off. In the pictures below we show a comparison of images when this filter is disabled and enabled.

Some cameras don't have Low-pass filter, that's why in Anyverse we provide the ability to turn it on or off.



Low-pass filter: off **Low-pass filter: on**

NEAR-INFRARED CUT-OFF FILTER

Camera sensors can catch a wider range of wavelengths than the human eye. It is well known that typical CMOS sensors are sensitive up to ~1000 nanometers. This filter allows removing those wavelengths, hence produced images are more natural to the human eye. In Anyverse™ solution, the electromagnetic spectrum that is simulated is in the range [380-1000] nanometers. The near-infrared (IR) filter can be configured to cut-off some specific spectrum range.

In the pictures below the IR filter affects the output images' final look. Notice how when the filter is enabled the reddish look in the image is removed, i. e. the filter removes part of the electromagnetic spectrum contributing to that red color.

IR filter: off

IR filter: on

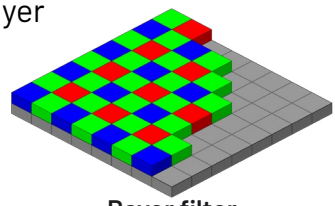
There are many different configurations and Anyverse can simulate any of them.

COLOR FILTER ARRAY

In the brief introduction to the sensor simulation pipeline, we mentioned that photodetectors only catch energy (light intensity), independently of the wavelength (color), so a sensor without a CFA will generate only gray-scale images. CFA filters differentiate wavelengths (colors), so the energy that each photodetector catches is only the energy coming from some specific wavelength (color). This is the sole purpose of the CFA, to provide color.

There are many different CFA configurations and Anyverse™ can simulate any of them. By default, we use the most commonly used filter: the Bayer filter. In the picture to the left we show the intensities reaching the sensor after the CFA.

Here, the typical Bayer filter configuration.



Bayer filter

FROM ENERGY TO VOLTAGE

> *If the core of the pipeline is the imaging sensor, the heart of the imaging sensor is the photodetector. In a nutshell, it collects photons and converts them into electrical current. To simulate it, the first thing we need to do is to convert the energy coming from the scene into photons.*

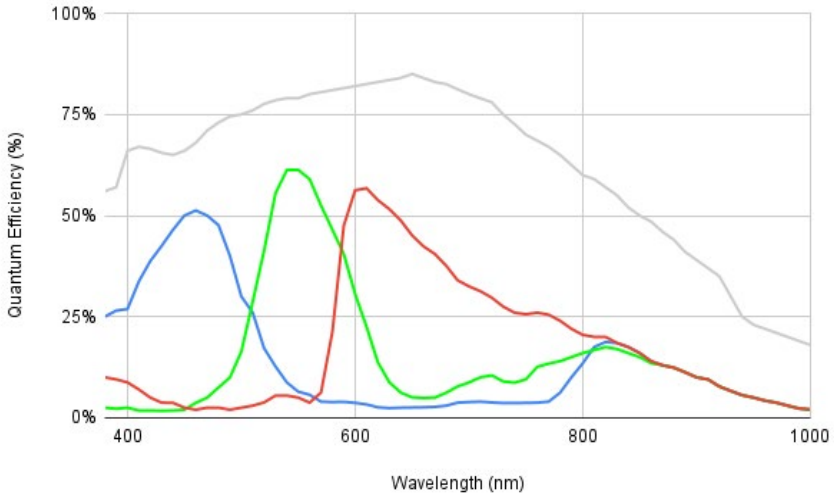
We do this using Planck's equation, which relates the amount of energy of a single photon with the photon wavelength ($E = hc/\lambda$). Because we know what is the amount of energy per wavelength coming from the scene, it's easy to compute the number of photons per wavelength.

Next, we have to transform the photons entering the photodetector into electrical current. This happens because the photons rip electrons from the sensor's silicon material into the conduction band producing the electrical current.

To calculate this we use the sensor's quantum efficiency (QE) curves. They provide the ratio between the photons entering the photodetector and the number of electrons that come out. This ratio depends on the wavelength (energy in the quantum world).

The QE curves are characteristic for every different sensor and depend on the materials, design and manufacturing process. Sensor manufacturers usually include them in the sensor specs datasheets.

Below is a graph of the QE curves for a SONY IMX556 sensor.



QE curves for a SONY IMX556 sensor [5]

Again, with quantum efficiency, we can see why it's so important that the rendering engine provides the scene's spectral radiance: No spectral radiance, no sensor simulation.

The QE curves are characteristic for every different sensor and depend on the materials, design and manufacturing of the sensor.

If the core of the pipeline is the imaging sensor, the heart of the imaging sensor is the photodetector.

On top of this, other sensor characteristics and parameters affect how the energy is transformed into a voltage.

OTHER SENSOR CHARACTERISTICS AND PARAMETERS

THE PIXEL SIZE

The larger the pixel size the larger the surface of the photodetector. This means that more photons enter the photodetector well, producing more electrons, providing a better dynamic range.

THE FILL FACTOR

The photodetector doesn't take up all the pixel's surface. This parameter represents the amount of the surface of the pixel occupied by the photodetector area.

THE EXPOSURE TIME

The longer the shutter is open, the more photons enter the photodetector well, producing more electrons, and a higher voltage when the shutter closes.

THE CONVERSION GAIN

This parameter describes how the electrons are converted into the output voltage. It's defined as the ratio between the photodetector's voltage swing and its well capacity. Sometimes manufacturers provide these two parameters instead of the conversion gain.

WELL CAPACITY

Is the maximum amount of electrons that the photodetector can produce. Larger photodetectors will have larger well capacity values, hence a larger output dynamic range.

VOLTAGE SWING

This is the photodetector's output voltage range.

PIXEL VIGNETTING

Now that we have the photodetector output voltage for every pixel, we are done with the heavy lifting of the simulation. It's time for the fine-tuning and we start with pixel vignetting.

Pixel vignetting is caused because a photodetector's efficiency depends on the angle of the light (energy) entering it.

Energy entering the photodetector with a steep angle produces a stronger signal than energy hitting it with a shallow angle. Notice that it seems reasonable to do this operation before the energy reaches the sensor, before the conversion to photons. But since it doesn't depend on the wavelength and the transformations from energy to final voltage are linear we can do the pixel vignetting on the voltage without loss of accuracy.

Pixel vignetting is caused because photodetector's efficiency depends on the angle of the light (energy) entering it.

Different parameters affect the amount of pixel vignetting:

- **The camera f-stop:** The larger the aperture, the larger the amount of energy that can reach the sensor perpendicular to it, hence less pixel vignetting and vice versa.
- **The focal length:** The larger the focal length the smaller the angle between the photodetectors in the sensor (pixels) and the optical center, hence less pixel vignetting and vice versa.

In this picture, we demonstrate how pixel vignetting affects the output image. We are using a focal length of 0.89 millimeters and a f-stop of 16.

Pixel vignetting

DARK CURRENT AND NOISE

Another extra fine-tuning effect is the dark voltage (current) we need to add to the sensor output voltage we just calculated. The name may seem daunting and mysterious, but it is just the residual voltage that we get when no photons are hitting the photodetectors, that's why it's called dark, which is the current in the sensor when there is no light.

Besides this intrinsic added current due to the circuitry in the sensor, now we have to introduce other sources of noise coming mostly from sensor's design and manufacturing particularities.

A quick note here: Like with pixel vignetting, it would be reasonable to introduce every type of noise when it happens. However, we introduce all of them after we get the sensor output voltage because they don't depend on the wavelength.

Original image without shot noise

For example, shot noise happens by random fluctuations in the number of photons entering the photodetector well. It seems reasonable to model this noise before converting the photons into voltage but because those fluctuations don't depend on the wavelength and the transformation from photon to voltage is linear, we can model this noise directly into the final voltage, in this particular case as a Poisson random distribution when the number of photons is low (low light conditions), a Gaussian random distribution otherwise.

Shot noise is more evident in the brightest parts of the image because it is proportional to the square root of the number of photons. In the pictures below we show how the shot noise affects the output image. Notice the image's reddish look because the IR cut-off filter was disabled, this is so for the sake of the noise visualization.

Shot noise amplified (x10) for the sake of the visualization

More structural noise, read noise comes from the electronic components of the sensor that are involved in the conversion of the photodetector charge into the final digital value. It depends only on the sensor design and its electronic components and it is affected by temperature.

See below a picture with this type of noise with an unusually high value just for the sake of visualization.

Read noise amplified for the sake of the visualization

Dark signal non-uniformity (DSNU) and photo-response non-uniformity (PRNU) are two different sources of noise with a common particularity. Both are fixed pattern noises.

Dark signal non-uniformity (DSNU) and photo-response non-uniformity (PRNU) are two different sources of noise with a common particularity. Both are fixed pattern noises.

PRNU comes from the fact that not all photodetectors in the sensor have the exact same area. This is because of the manufacturing process and it's different for every unit manufactured.

As a curiosity, this PRNU noise can be used in forensic science to identify the camera that took a photo. It is proportional to the energy reaching the sensor.

DSNU in this case, comes from impurities introduced in the photodetectors materials during the manufacturing process. We saw that the dark signal is the amount of current that is present in the circuitry when there is no energy reaching the photodetectors.

DSNU says that this current is not uniform because every photodetector might have small variations in the material used for their manufacturing.

See a picture below with simulated DSNU and PRNU noise. Again the noise values used are unusually amplified for the sake of visualization.

DSNU + PRNU noise amplified for the sake of the visualization

As a curiosity, this PRNU noise can be used in forensic science to identify the camera that took a photo. It is proportional to the energy reaching the sensor.

ANALOG GAIN AND DIGITAL CONVERSION

Now we have a "noisy", but realistic signal ready to convert into a digital value. But before that, it's time for analog gain. This is an amplification of the voltage coming from the photodetectors before we do the final analog to digital conversion. We can apply an analog offset as well, which can be useful to define the black level of the output image.

The final step of the imaging sensor simulation pipeline is the analog to digital converter. It converts the voltage to a digital value that finally represents the color captured by the sensor. The number of bits used to encode the analog value is a parameter of this conversion stage. The more bits we use, the more colors we can represent in the final image.

EXPOSURE

One last thing to consider about the imaging sensor: the effect of the exposure and the shutters.

The exposure is the time the photodetectors are active. The longer they are active the more photons will be collected, hence more intensity on the final image.

To control the exposure, CMOS sensors use electronic shutters, this means that there is circuitry that controls when and for how long the photodetectors are collecting photons.

CCD sensors use electronic shutters as well but there is a difference. CCD sensors expose all photodetectors at the same time and for the same amount of time.

Every photodetector sees the same point in time as the others. This is called global shutter. In contrast, CMOS sensors expose rows of photodetectors sequentially, this is called rolling shutter.

The CMOS circuitry is less complex than the CCD circuitry and this is a key factor to consider when going to very high sensor resolutions.

The cost is the main reason why most of the sensors used in the AV industry today use CMOS technology.

The disadvantage of rolling shutter versus global shutter is the distortion introduced in the images caused by the different exposure times for different rows.

In Anyverse™ we can simulate both, global and rolling shutters.

In the image below you can see a comparison.

The only object moving in that scene is the vehicle. You clearly can see how the vehicle is distorted because of the row exposure sequence in the case of the rolling shutter.

The advantage of rolling shutter versus global shutter is the production cost.

The only object moving in that scene is the vehicle. You clearly can see how the vehicle is distorted because of the row exposure sequence in the case of the rolling shutter.

Rolling shutter

Global shutter

We also implement an auto-exposure feature. To implement it we run the pipeline using the maximum energy coming from the scene.

With this information, we compute an exposure value that provides the best mapping between the energy levels and the voltage level at the output of the photodetectors. And that puts the cherry on how we simulate the imaging sensor. We have followed the light on its journey through the lens and the sensor.

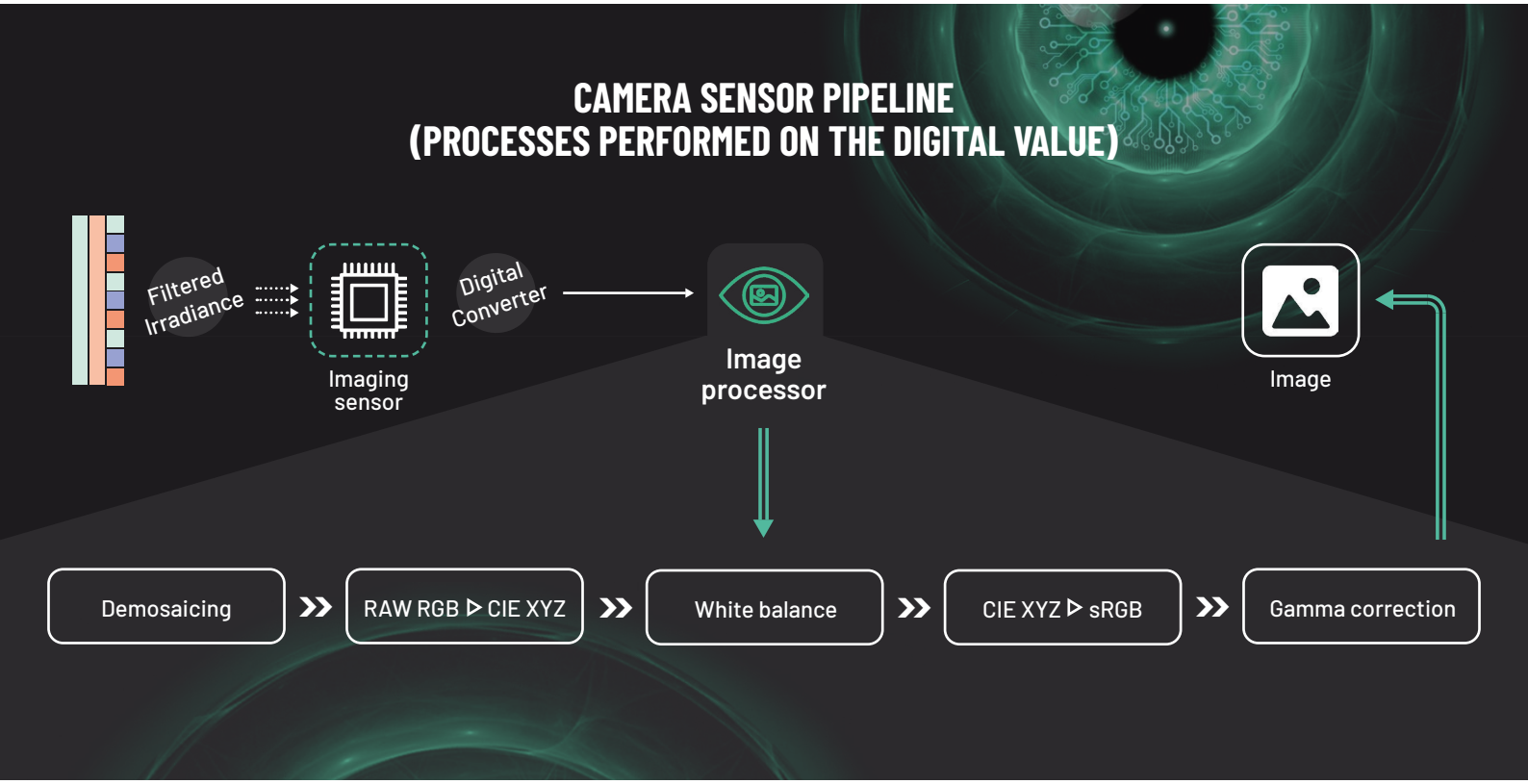
All the processes described in this chapter happen for every photodetector (pixel) in the sensor, ending up with the digital RAW value for every pixel. Now a new journey starts for these values until we get a useful image.

3.3 THE IMAGE PROCESSOR

> *To consume the RAW digital data coming from the sensor as a beautiful color image, we need to process it and adapt it to what the human eye sees and what the different devices can display. This last part of the pipeline is known as the ISP (image signal processor). It is responsible for converting the digital values coming out from the sensor to the final color.*

With Anyverse™ we provide the RAW data, before any ISP process, in case you want to apply your own ISP. ISPs are the camera manufacturer's "secret sauce", because with this process they can provide that "special look" they want to the final pictures.

Here you have the different processes and transformations that the ISP does on the digital values:



Demosaicing is the process of reconstructing the color per pixel given the fact that during the capture process we used color filters. As explained earlier, every photodetector is capturing only one specific color. The image coming from the sensor, also known as RAW data, is a grayscale image that has to be processed to compute a color image.

In the picture below you can see a RAW image with a 12-bit depth resulting from the imaging sensor pipeline. This is the input to the ISP.

Raw image 12-bit depth

The reconstructed image after the demosaicing process is already a color image. This color image is the representation of the image as seen by the imaging sensor using the spectral sensitivities given by the color filter array.

However, the spectral sensitivities of the human eye are different, so we need to convert the color from the **raw RGB color space** to the **CIE XYZ color space**.

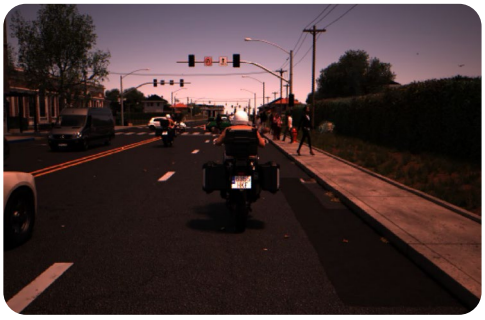
The CIE color model is a color space model created by the International Commission on Illumination known as the Commission Internationale de l'Eclairage (CIE). It is also known as the CIE XYZ color space or the CIE 1931 XYZ color space.

The CIE color model is a color space model created by the International Commission on Illumination known as the Commission Internationale de l'Eclairage (CIE).

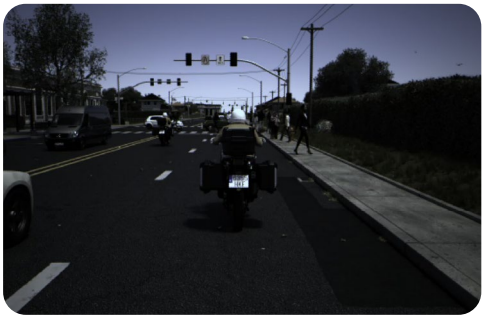
The CIE color model is a mapping system that uses tristimulus (a combination of 3 color values that are close to red/green/blue) values, which are plotted on a 3D space. When these values are combined, they can reproduce any color that a human eye can perceive.

The CIE specification is supposed to be able to accurately represent every single color the human eye can perceive.

Since two images are worth 2,000 words, I think I got my math right, see below for a comparison between a raw RGB image and a CIE XYZ image.



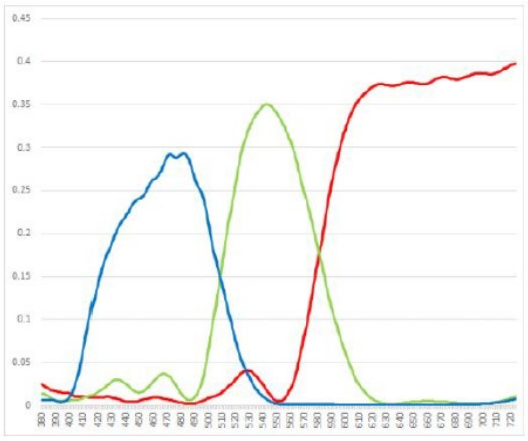
Raw RGB
(How the camera sees the world)



CIE XYZ
(How the human sees the world)

In the comparison above, notice the reddish tone of the image as seen by the camera sensor. This is because even though the IR cut-off filter is removing the range above 700 nm, the red filter still has a wider range of wavelengths than the green and blue filters.

Below the Spectral Power Distribution (SPD) for the RGB filters used to produce all the images throughout this eBook.



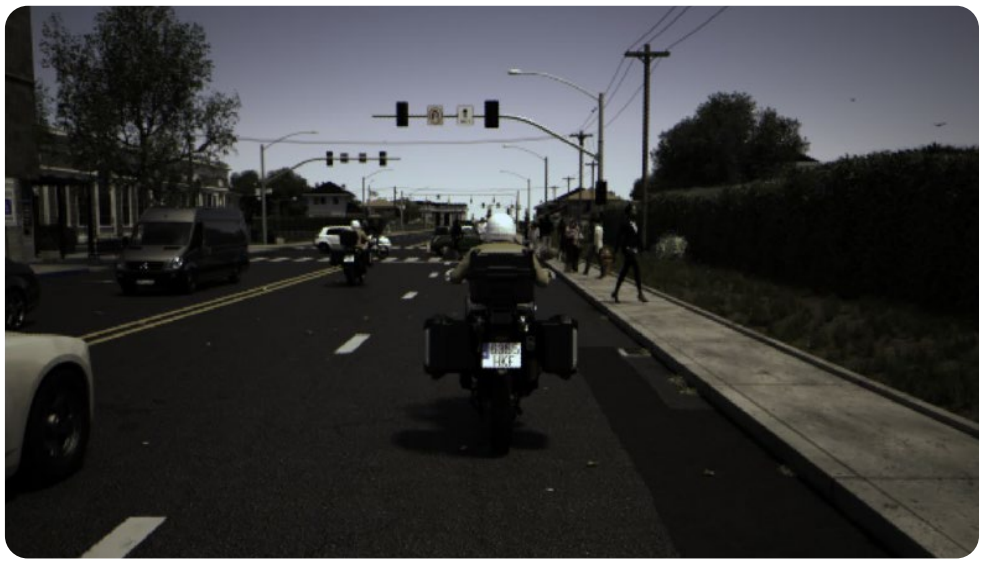
RGB filters

Something great about the human eye is that under different lighting conditions, it always perceives the white color (and others) correctly. Cameras can't do that, they have to compute the colors precisely by their spectral profile.

We haven't finished yet. Something great about the human eye is that under different lighting conditions, it always perceives the white color (and others) correctly. Cameras can't do that, they have to compute the colors precisely by their spectral profile.

Somehow we have to process those colors so they look correct for the human eye, this process is known as white balance. There are different approaches to do it.

Below, for example, the CIE XYZ image white balanced using the gray world approach, where we force the average color in the scene to be gray.



White balance (gray world)

Whereas in this other one, we used the white world approach, where we force the brightest color in the scene to be white.

White balance (white world)

Like the human eye, every device has specific colors they can display, they have their own color space. So we need to convert the image from the CIE XYZ color into the display device color space.

To summarize, now we have a white balanced image in the human eye reference color space: CIE XYZ. We may think we are finished but the problem is that we don't consume the digital image directly, we need to print it or display it on a device. Like the human eye, every device has specific colors they can display, they have their own color space.

So we need to convert the image from the CIE XYZ color into the display device color space. The question is: what color should we send to the display device so the represented colors match the CIE XYZ colors?

There are many target device color spaces, sRGB, Adobe RGB 98, ECI RGB, PAL/SECAM, NTSC 1953, etc. With Anyverse™ you can simulate any of them, the one that is most widely used for display devices such as computer, phone or tablet screens is the sRGB color space.

Below the final image using the sRGB color space.

sRGB

There are many target device color spaces, sRGB, Adobe RGB 98, ECI RGB, PAL/SECAM, NTSC 1953, etc. With Anyverse™ you can simulate any of them.

A bit dark isn't it? That's because we haven't applied gamma correction yet.

The way the human eye perceives brightness is not linear, this makes it more sensitive to dark tones. That's why images without gamma correction look very dark. Gamma correction cancels out the display response curve effectively increasing the brightness of the final image.

Below the image with gamma correction applied.

Gamma correction

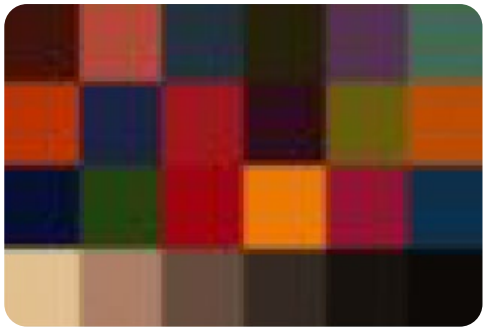
Gamma correction cancels out the display response curve effectively increasing the brightness of the final image.

4. VALIDATION

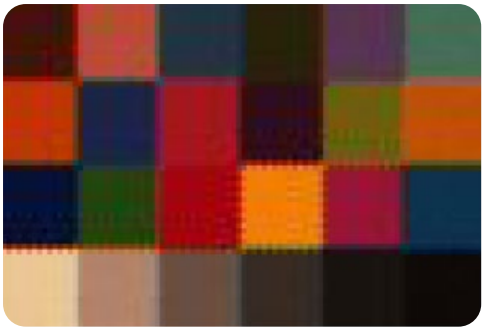
> *You may ask, how accurate or close to real sensors is the Anyverse™ sensor simulation pipeline implementation described here? Fair enough. In this chapter, we describe the validation method we have run to clear any doubts.*

We have validated the Anyverse™ sensor simulation pipeline using the ISET Toolbox for Matlab from Stanford University. This toolbox allows for the simulation of a camera sensor and has been validated by comparing simulated data with real data [5][6].

We used the ISET toolbox to obtain the colors from the Macbeth color checker[7] using some specific sensor configuration. We used the same sensor configuration to obtain the colors using Anyverse™ 's sensor simulation. Below the images obtained from the two simulators.



Macbeth chart (ISET toolbox)



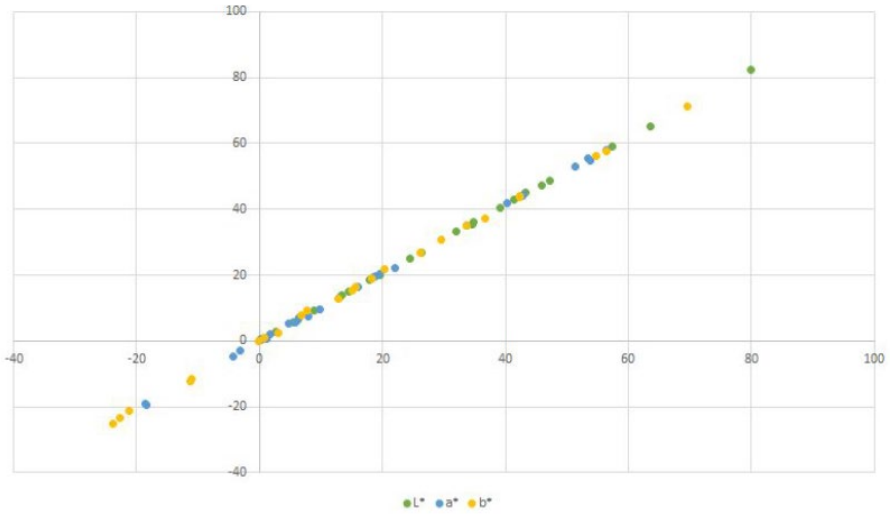
Macbeth chart (Anyverse)

In the table below we show the colors using the CIELAB color space for the macbeth's 24 patches from the two simulators. Notice how they are in good agreement.

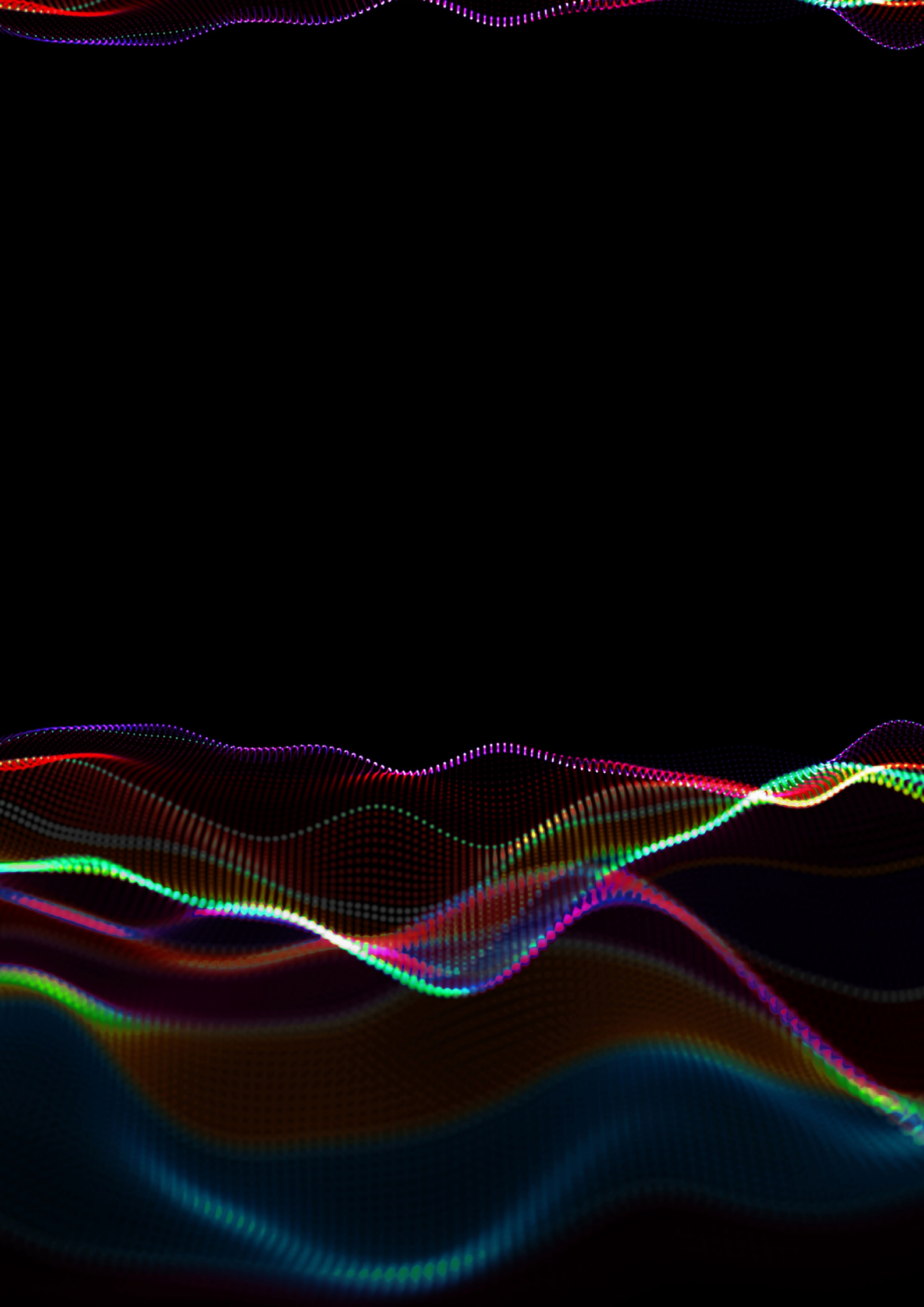
L*	a*	b*	L*	a*	b*
13.4	22	15.6	14	22.3	16.5
45.9	42.7	33.6	47.3	44.1	34.9
19.5	-3.1	-11	20	-2.9	-11.6
13	0.3	15	13.1	0.8	15.2
26.4	19.5	-11.2	27	20.2	-12.2
41.4	-18.4	7.8	42.8	-19.5	9.3
43.2	53.5	54.8	45	55.4	56.3
15.5	5.9	-22.8	15	5.8	-23.5
34.7	53.8	36.7	35.+6	54.9	37.3
8.9	18.8	-0.1	9.3	19.8	-0.1
39.1	0.2	42.4	40.4	0.4	44
47.3	42.9	56.4	48.7	44.3	57.5
5.5	6.4	-23.8	5.7	7.2	-25
24.4	-18.6	26.1	25.2	-19.1	26.9
33.6	56.4	42.3	35	58	43.5
63.6	40.2	69.6	65.3	41.9	71.2
31.9	51.4	18.3	33.3	52.9	19.1
18.5	-4.3	-21.3	19.4	-4.6	-21.3
79.9	7.9	29.6	82.4	7.6	30.9
57.4	16	20.4	59.1	16.6	21.7
34.9	9.8	12.8	36.1	9.7	12.7
17.8	4.8	6.8	18.4	5.2	7.7
6.3	1.8	3	6.3	1.9	2.5
2.7	1.1	0.8	2.9	0.7	1.1

On the left CIELAB values from ISET toolbox, on the right values from Anyverse.

In the chart below we show a scatter plot for the color values from the two simulators. The trend shows that the results are very similar.



Scatter plot for CIELAB values



5. CONCLUSION

> *If there is only one thing I would like you to take away from this eBook is that to do an accurate sensor simulation you need the light coming from the scene described using its electromagnetic spectrum profile. It's the only way to simulate the physical phenomena happening on the optical system and the sensor. No light, no simulation, is that simple.*

And no simulation means your synthetic data may not be that useful to train and test deep learning-based perception systems. It may be more difficult for the neural networks to generalize to real-world images. Because at the end of the day, that is every perception system's goal: understand the real world and interpret it.

That's is exactly what Anyverse™ wants to accomplish. And we do it by recreating the real world you need in realistic, varied datasets using our own physically-based render engine and the sensor simulation pipeline explained in this eBook you are about to finish.

Imagine you can generate as many images as you need for your training and testing, automatically annotated with accurate ground truth data, faithfully simulating the exact cameras you rig in your real system. How powerful is that?

Say that you want to try new cameras on your system and the impact this change may have in the overall system. Doing this without sensor simulation means: rigging the new cameras on the real system and taking it out to collect data, curate it and annotate it, train the system and see what happens. What's your flexibility to tweak camera parameters? Zero.

With Anyverse™ sensor simulation along with the data generation platform, you can run as many experiments as you want, changing camera parameters in every iteration to see how they affect neural network performance. Don't let the data generation pipeline be an independent process in your system development.

But sensor simulation goes beyond data. If you are developing your own sensors you can make design decisions without the complexity and cost of prototyping on silicon.

Sensor simulation goes beyond data. If you are developing your own sensors you can make design decisions without the complexity and cost of prototyping on silicon.

You can develop the best "eye-brain" combination for your perception problem without leaving the lab. It allows efficient agile practices from classic software development applied to software 2.0 development, a term coined by Andrej Karpathy in 2017 describing a paradigm change when developing deep learning-based systems[8].

I hope you enjoyed and learnt reading this, I did putting it together for you.

Stay tuned for more content from us. In the meantime follow us on our social networks and don't hesitate to contact us if you have any questions about sensor simulation or anything else.

Special thanks to Javier Salado, Technical Product Manager, and Carlos Luque, Marketing Manager for working hard behind the scenes to put all pieces together to produce this eBook.

Ángel Tena | Chief Technology Officer

REFERENCES

1. Carlson A., Skinner K. A., Vasudevan R., Johnson-Roberson M.: Modeling Camera Effects to Improve Visual Learning from Synthetic Data. arXiv preprint arXiv:1803.07721v6 (2018) https://arxiv.org/abs/1803.0772

2. Liu Z., Lian T., Farrell J.E., Wandell B.A.: Neural Network Generalization: The impact of camera parameters. arXiv preprint arXiv: 1912.03604v1 (2019) https://arxiv.org/abs/1912.03604

3. https://www.cambridgeincolour.com/tutorials/camera-sensors.htm

4. https://en.wikipedia.org/wiki/Moir%C3%A9_pattern

5. https://thinklucid.com/product/triton-89-mp-imx267/

6. Farrell J.E., Wandell B.A. (2014) Handbook of digital imaging - Image Systems Simulation, 373-401

7. https://en.wikipedia.org/wiki/ColorChecker

8. Software 2.0, Andrej Karpathy. Medium article. https://karpathy.medium.com/software-2-0-a64152b37c35

ANYVERSE

DON'T RISK YOUR COMPUTER VISION SYSTEM PERFORMANCE

Not all synthetic images are valid for deep learning model training.

Anyverse sensor simulation physically simulates optics and sensors to generate high-fidelity datasets, reducing the domain gap to accurately generalize to real-world inputs.

GET STARTED

WITH ANYVERSE SYNTHETIC DATA PLATFORM

Ask our team for a demo, or download the platform for free to start generating data yourself.

ASK FOR A DEMO

DOWNLOAD PLATFORM

Click **here** to access Anyverse's quick tutorials for new users

ANYVERSE

Anyverse is a scalable, cloud-based synthetic data platform that accelerates the development of computer vision-based solutions for autonomous applications capable of covering and supplying all the data needs throughout the entire development cycle. From the initial stages of design or prototyping, through training/testing, and ending with the "fine-tuning" of the system to maximize its capabilities and performance.

Anyverse works with bluechip companies worldwide and offers unique solutions for several markets such as in-cabin monitoring, ADAS, self-driving, security, defense, or inspection autonomous systems, among others.

CONNECT WITH US!

Our social media channels help you to stay up to date on our latest insights, news and more.

anyverse.ai

linkedin.com/company/anyverse-ai

@AnyverseAI

youtube.com/@anyverse_AI

@anyverse.ai

medium.com/anyverse